

HOTEL RESERVATION AND GUEST SERVICE MANAGEMENT SYSTEM

1. COVER PAGE

Project Title:

Hotel Reservation and Guest Service Management System

Course: Database Management Systems (DBMS)

Submitted By:

Name: Kavuri Rohan Chowdary

Roll Number: 25WU0102122

Section: CSE AI&ML PANTHERS

Faculty Name: Dr. Kiran Mayee Adavala

Department: AI&ML

Academic Year: 2025–2029

University/Institution: WOXSSEN UNIVERSITY

2. ABSTRACT

The Hotel Reservation and Guest Service Management System is an application developed on a database to help manage hotel reservations, guest information, rooms, rate plans, stays, guest services, housekeeping tasks, service charges, invoices, and payments.

The system's primary aim is to automate hotel record management by replacing it with a centralised relational database. The database is built on the MySQL database management system and allows users to insert, view, update, and delete records through a user interface.

The database contains eleven major entities: RoomTypes, Rooms, RatePlans, Guests, Reservations, Stays, ServiceRequests, HousekeepingTasks, ServiceCharges, Invoices, and Payments. Primary keys and foreign keys have been used to ensure that there are relationships between these tables and to maintain data integrity.

The application also provides SQL queries that make use of SELECT, INSERT, UPDATE, DELETE, JOIN, GROUP BY, COUNT, and SUM operations.

3. INTRODUCTION AND PROBLEM STATEMENT

3.1 Introduction

Hotels generate lots of information on guests, rooms, bookings, payments, services, housekeeping, and invoices. Manually managing such information can be complicated since the data may have duplicates, become lost, or get altered.

The proposed Hotel Reservation and Guest Service Management System is a centralized database that will store hotel-related information.

Hotel staff can perform the following actions using the system:

Managing room types.

Managing rooms.

Managing rate plans.

Registering guests.

Creating and managing bookings.

Managing guest stays.

Managing services requests.

Managing housekeeping tasks.

Charging for services.

Generating invoices.

Recording payments.

The system utilizes a relational database which consists of connected tables via primary and foreign keys.

3.2 Problem Statement

Using traditional or manual hotel management systems can cause the following problems:

Guest record duplicates.

Difficulty tracking the availability of the rooms.

Errors in reservation records.

Difficulty tracking guest services.

Manually calculating service charges.

Difficulty managing invoices and payments.

Slow access to information.

Data inconsistency.

Difficulty generating reports.

Therefore, a centralized DBMS-based system is necessary to store, retrieve, update, and manage hotel reservation and guest service information.

4. OBJECTIVES AND SCOPE

4.1 Objectives

The major objectives of the project are:

Designing a relational database for hotel management.

Storing guest information systematically.

Managing different room types and rooms.

Managing hotel rate plans.

Creating and managing bookings.
Managing guest stays.
Managing guest service requests.
Maintaining housekeeping tasks.
Calculating and storing service charges.
Generating and managing invoices.
Recording and managing payments.
Performing CRUD operations via user interface.
Maintaining data integrity using primary and foreign keys.
Demonstrating SQL queries and relational operations.
Reducing data redundancy and increasing data consistency.

4.2 Scope

The system can be used by hotel staff to manage daily hotel operations.

The scope of the system is:

Guest registration.
Room management.
Room type management.
Booking management.
Stay management.
Guest service management.
Housekeeping management.
Billing and invoice management.
Payment management.

Records search and viewing.

Records insertion and deletion.

SQL query execution.

Further system updates can include online booking, customer login, payment gateway, email notifications, and mobile application development.

5. SOFTWARE AND HARDWARE REQUIREMENTS

5.1 Software Requirements

Software	Purpose
HTML	Structure for web pages
CSS	User Interface styling
JavaScript	Frontend interactions
Node.js	Back-end execution platform
Express.js	Back-end application framework
MySQL	Relational Database
MySQL Workbench	SQL database design
Replit	Development and deployment platform
Web Browser	Application Execution Platform

5.2 Hardware Requirements

Component	MINIMUM REQUIREMENT
CPU	Intel Core I3 or equivalent
RAM	4GB or above
Storage	10GB Free Space
Internet	Required for Replit/cloud deployment

Display 1366 × 768 or above

Input Keyboard and Mouse

6. ER DIAGRAM

The Entity Relationship Diagram shows entities & relationships in the Hotel Reservation and Guest Service Management System.

MAIN ENTITIES:

RoomTypes

Rooms

RatePlans

Guests

Reservations

Stays

ServiceRequests

HousekeepingTasks

ServiceCharges

Invoices

Payments

MAIN RELATIONSHIPS

1 RoomType can have many Rooms.

1 Guest can have many Reservations.

1 Room can have many Reservations through Time.

1 Rate Plan can have many Reservations.

1 Reservation can have 1 Stay.

1 Guest can make multiple ServiceRequests.

- 1 Stay can have many ServiceRequests.
- 1 Room can have many HousekeepingTasks.
- 1 Stay can have many ServiceCharges.
- 1 ServiceRequest can have ServiceCharges.
- 1 Stay can have one Invoice.
- 1 Invoice can have many Payments.

ER Diagram Relationship Summary

RoomTypes (1) → (M) Rooms

Guests (1) → (M) Reservations

Rooms (1) → (M) Reservations

RatePlans (1) → (M) Reservations

Reservations (1) → (1/M) Stays

Guests (1) → (M) ServiceRequests

Stays (1) → (M) ServiceRequests

Rooms (1) → (M) HousekeepingTasks

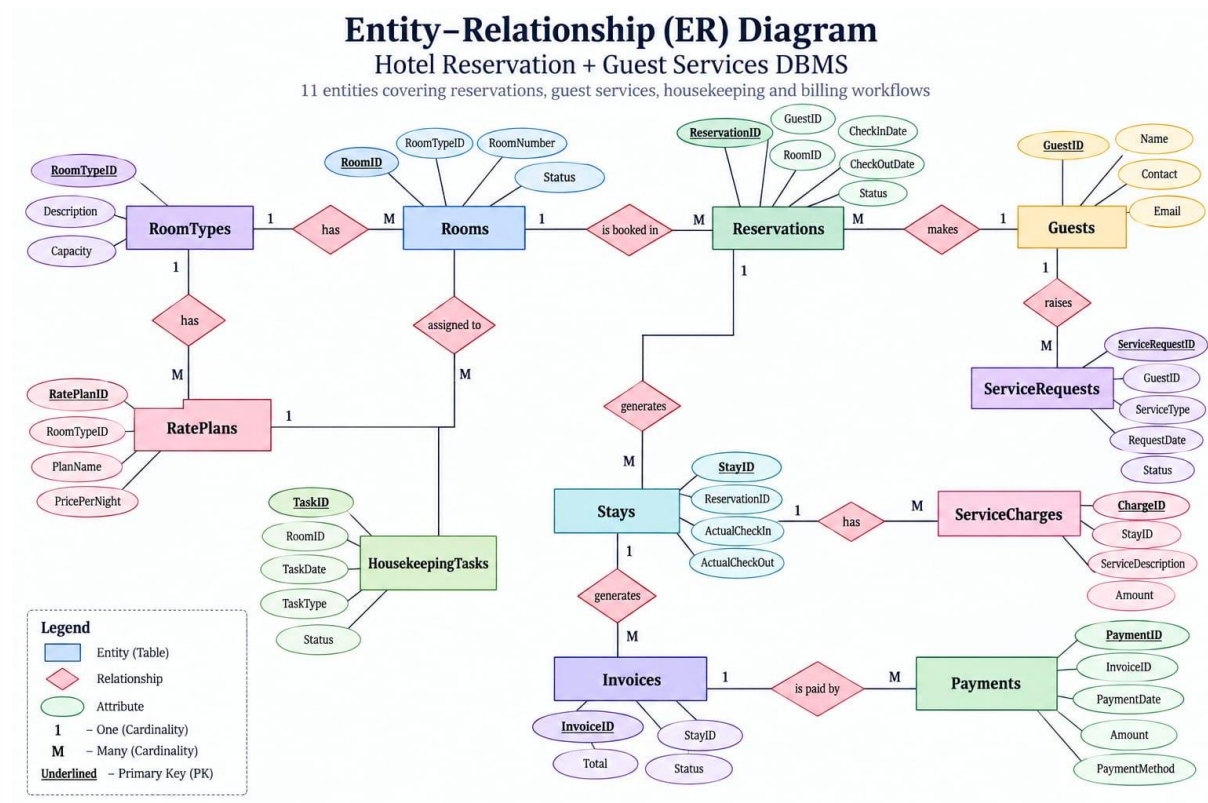
Stays (1) → (M) ServiceCharges

ServiceRequests (1) → (M) ServiceCharges

Stays (1) → (1) Invoices

Invoices (1) → (M) Payments

INSERT ER DIAGRAM GRAPHIC HERE.



```
CREATE DATABASE HotelManagement;

USE HotelManagement;

CREATE TABLE RoomTypes (
    RoomTypeID INT PRIMARY KEY AUTO_INCREMENT,
    TypeName VARCHAR(50) NOT NULL,
    Description VARCHAR(255),
    Capacity INT NOT NULL,
    BasePrice DECIMAL(10,2) NOT NULL
);

CREATE TABLE Rooms (
    RoomID INT PRIMARY KEY AUTO_INCREMENT,
    RoomNumber VARCHAR(10) UNIQUE NOT NULL,
    RoomTypeID INT NOT NULL,
    Floor INT NOT NULL,
    Status VARCHAR(20) NOT NULL,
```

```
FOREIGN KEY (RoomTypeID)
    REFERENCES RoomTypes(RoomTypeID)
);

CREATE TABLE RatePlans (
    RatePlanID INT PRIMARY KEY AUTO_INCREMENT,
    PlanName VARCHAR(50) NOT NULL,
    Description VARCHAR(255),
    PricePerNight DECIMAL(10,2) NOT NULL,
    CancellationPolicy VARCHAR(255)
);

CREATE TABLE Guests (
    GuestID INT PRIMARY KEY AUTO_INCREMENT,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Phone VARCHAR(15) NOT NULL,
    Address VARCHAR(255),
    IDProof VARCHAR(50)
);

CREATE TABLE Reservations (
    ReservationID INT PRIMARY KEY AUTO_INCREMENT,
    GuestID INT NOT NULL,
    RoomID INT NOT NULL,
    RatePlanID INT NOT NULL,
    ReservationDate DATE NOT NULL,
    CheckInDate DATE NOT NULL,
```

CheckOutDate DATE NOT NULL,
NumberOfGuests INT NOT NULL,
Status VARCHAR(20) NOT NULL,

FOREIGN KEY (GuestID)
REFERENCES Guests(GuestID),

FOREIGN KEY (RoomID)
REFERENCES Rooms(RoomID),

FOREIGN KEY (RatePlanID)
REFERENCES RatePlans(RatePlanID)

);

CREATE TABLE Stays (
StayID INT PRIMARY KEY AUTO_INCREMENT,
ReservationID INT NOT NULL,
ActualCheckIn DATETIME,
ActualCheckOut DATETIME,
NumberOfNights INT,
StayStatus VARCHAR(20),

FOREIGN KEY (ReservationID)
REFERENCES Reservations(ReservationID)

);

CREATE TABLE ServiceRequests (
ServiceRequestID INT PRIMARY KEY AUTO_INCREMENT,

GuestID INT NOT NULL,
StayID INT NOT NULL,
ServiceType VARCHAR(100) NOT NULL,
RequestDate DATE NOT NULL,
RequestTime TIME,
Status VARCHAR(20) NOT NULL,
SpecialInstructions VARCHAR(255),

FOREIGN KEY (GuestID)
REFERENCES Guests(GuestID),

FOREIGN KEY (StayID)
REFERENCES Stays(StayID)

);

CREATE TABLE HousekeepingTasks (
TaskID INT PRIMARY KEY AUTO_INCREMENT,
RoomID INT NOT NULL,
TaskDate DATE NOT NULL,
TaskType VARCHAR(50) NOT NULL,
AssignedStaff VARCHAR(100),
Status VARCHAR(20) NOT NULL,
Remarks VARCHAR(255),

FOREIGN KEY (RoomID)
REFERENCES Rooms(RoomID)

);

```
CREATE TABLE ServiceCharges (  
    ServiceChargeID INT PRIMARY KEY AUTO_INCREMENT,  
    StayID INT NOT NULL,  
    ServiceRequestID INT,  
    ServiceName VARCHAR(100) NOT NULL,  
    Quantity INT NOT NULL,  
    UnitPrice DECIMAL(10,2) NOT NULL,  
    TotalAmount DECIMAL(10,2) NOT NULL,  
    ChargeDate DATE NOT NULL,  
  
    FOREIGN KEY (StayID)  
        REFERENCES Stays(StayID),  
  
    FOREIGN KEY (ServiceRequestID)  
        REFERENCES ServiceRequests(ServiceRequestID)  
);
```

```
CREATE TABLE Invoices (  
    InvoiceID INT PRIMARY KEY AUTO_INCREMENT,  
    StayID INT NOT NULL,  
    InvoiceDate DATE NOT NULL,  
    RoomCharges DECIMAL(10,2) NOT NULL,  
    ServiceCharges DECIMAL(10,2) NOT NULL,  
    TaxAmount DECIMAL(10,2) NOT NULL,  
    TotalAmount DECIMAL(10,2) NOT NULL,  
    InvoiceStatus VARCHAR(20) NOT NULL,
```

```

FOREIGN KEY (StayID)
    REFERENCES Stays(StayID)
);

CREATE TABLE Payments (
    PaymentID INT PRIMARY KEY AUTO_INCREMENT,
    InvoiceID INT NOT NULL,
    PaymentDate DATE NOT NULL,
    Amount DECIMAL(10,2) NOT NULL,
    PaymentMethod VARCHAR(30) NOT NULL,
    PaymentStatus VARCHAR(20) NOT NULL,
    TransactionReference VARCHAR(100),

    FOREIGN KEY (InvoiceID)
        REFERENCES Invoices(InvoiceID)
);

INSERT INTO RoomTypes
(TypeName, Description, Capacity, BasePrice)
VALUES
('Standard', 'Comfortable standard room', 2, 2500.00),
('Deluxe', 'Spacious deluxe room', 2, 3500.00),
('Suite', 'Luxury suite with living area', 4, 6000.00),
('Family', 'Large room suitable for families', 4, 5000.00),
('Executive', 'Premium executive room', 2, 4500.00);

INSERT INTO Rooms
(RoomNumber, RoomTypeID, Floor, Status)
VALUES

```

('101', 1, 1, 'Available'),

('102', 1, 1, 'Occupied'),

('103', 2, 1, 'Available'),

('104', 2, 1, 'Cleaning'),

('201', 3, 2, 'Available'),

('202', 3, 2, 'Occupied'),

('203', 4, 2, 'Available'),

('204', 4, 2, 'Available'),

('301', 5, 3, 'Occupied'),

('302', 5, 3, 'Available');

INSERT INTO RatePlans

(PlanName, Description, PricePerNight, CancellationPolicy)

VALUES

('Standard Plan', 'Room only', 2500.00, 'Free cancellation 24 hours before check-in'),

('Breakfast Plan', 'Room with breakfast', 3000.00, 'Free cancellation 24 hours before check-in'),

('Premium Plan', 'Room with breakfast and dinner', 4500.00, 'Free cancellation 48 hours before check-in'),

('Family Plan', 'Family package', 5500.00, 'Free cancellation 48 hours before check-in'),

('Luxury Plan', 'Premium luxury package', 7000.00, 'Non-refundable');

INSERT INTO Guests

(FirstName, LastName, Email, Phone, Address, IDProof)

VALUES

('Aarav', 'Sharma', 'aarav@gmail.com', '9876543210', 'Hyderabad', 'Aadhar'),

('Ananya', 'Reddy', 'ananya@gmail.com', '9876543211', 'Secunderabad', 'Aadhar'),

('Rahul', 'Kumar', 'rahul@gmail.com', '9876543212', 'Bangalore', 'Passport'),
('Priya', 'Singh', 'priya@gmail.com', '9876543213', 'Mumbai', 'Aadhar'),
('Arjun', 'Rao', 'arjun@gmail.com', '9876543214', 'Chennai', 'Passport'),
('Sneha', 'Patel', 'sneha@gmail.com', '9876543215', 'Pune', 'Aadhar'),
('Vikram', 'Das', 'vikram@gmail.com', '9876543216', 'Kolkata', 'Passport'),
('Kavya', 'Nair', 'kavya@gmail.com', '9876543217', 'Kochi', 'Aadhar'),
('Rohit', 'Verma', 'rohit@gmail.com', '9876543218', 'Delhi', 'Aadhar'),
('Meera', 'Iyer', 'meera@gmail.com', '9876543219', 'Hyderabad', 'Passport');

INSERT INTO Reservations

(GuestID, RoomID, RatePlanID, ReservationDate,
CheckInDate, CheckOutDate, NumberOfGuests, Status)

VALUES

(1, 1, 1, '2026-10-01', '2026-10-10', '2026-10-12', 2, 'Confirmed'),
(2, 3, 2, '2026-10-02', '2026-10-11', '2026-10-14', 2, 'Confirmed'),
(3, 5, 3, '2026-10-03', '2026-10-15', '2026-10-18', 3, 'Confirmed'),
(4, 7, 4, '2026-10-04', '2026-10-16', '2026-10-20', 4, 'Confirmed'),
(5, 9, 5, '2026-10-05', '2026-10-18', '2026-10-20', 2, 'Confirmed'),
(6, 2, 1, '2026-10-05', '2026-10-21', '2026-10-23', 1, 'Confirmed'),
(7, 4, 2, '2026-10-06', '2026-10-22', '2026-10-25', 2, 'Pending'),
(8, 8, 3, '2026-10-06', '2026-10-24', '2026-10-27', 3, 'Confirmed'),
(9, 10, 5, '2026-10-07', '2026-10-26', '2026-10-28', 2, 'Pending'),
(10, 6, 4, '2026-10-07', '2026-10-28', '2026-10-31', 4, 'Confirmed');

INSERT INTO Stays

(ReservationID, ActualCheckIn, ActualCheckOut,
NumberOfNights, StayStatus)

VALUES

```
(1, '2026-10-10 14:00:00', '2026-10-12 11:00:00', 2, 'Completed'),  
(2, '2026-10-11 14:30:00', NULL, 3, 'Active'),  
(3, '2026-10-15 13:45:00', NULL, 3, 'Active'),  
(4, '2026-10-16 14:00:00', NULL, 4, 'Active'),  
(5, '2026-10-18 15:00:00', NULL, 2, 'Active');
```

```
INSERT INTO ServiceRequests
```

```
(GuestID, StayID, ServiceType, RequestDate,  
RequestTime, Status, SpecialInstructions)
```

```
VALUES
```

```
(1, 1, 'Room Service', '2026-10-10', '19:30:00',  
'Completed', 'Dinner for two'),
```

```
(2, 2, 'Laundry', '2026-10-11', '10:00:00',  
'Pending', 'Express laundry'),
```

```
(3, 3, 'Extra Bed', '2026-10-15', '16:00:00',  
'Completed', 'Extra bed required'),
```

```
(4, 4, 'Airport Pickup', '2026-10-16', '09:00:00',  
'Pending', 'Pickup from airport'),
```

```
(5, 5, 'Room Service', '2026-10-18', '20:00:00',  
'Completed', 'Vegetarian dinner');
```

```
INSERT INTO HousekeepingTasks
```

```
(RoomID, TaskDate, TaskType, AssignedStaff, Status, Remarks)
```

```
VALUES
```

(1, '2026-10-10', 'Cleaning', 'Ramesh', 'Completed', 'Room cleaned'),
(2, '2026-10-11', 'Cleaning', 'Suresh', 'Pending', 'Regular cleaning'),
(3, '2026-10-11', 'Laundry', 'Anil', 'Completed', 'Bed linen replaced'),
(4, '2026-10-12', 'Cleaning', 'Ramesh', 'Pending', 'Deep cleaning'),
(5, '2026-10-12', 'Inspection', 'Suresh', 'Completed', 'Room inspected');

INSERT INTO ServiceCharges

(StayID, ServiceRequestID, ServiceName,
Quantity, UnitPrice, TotalAmount, ChargeDate)

VALUES

(1, 1, 'Room Service', 2, 500.00, 1000.00, '2026-10-10'),
(2, 2, 'Laundry', 5, 100.00, 500.00, '2026-10-11'),
(3, 3, 'Extra Bed', 1, 800.00, 800.00, '2026-10-15'),
(4, 4, 'Airport Pickup', 1, 1200.00, 1200.00, '2026-10-16'),
(5, 5, 'Room Service', 2, 450.00, 900.00, '2026-10-18');

INSERT INTO HousekeepingTasks

(RoomID, TaskDate, TaskType, AssignedStaff, Status, Remarks)

VALUES

(1, '2026-10-10', 'Cleaning', 'Ramesh', 'Completed', 'Room cleaned'),
(2, '2026-10-11', 'Cleaning', 'Suresh', 'Pending', 'Regular cleaning'),
(3, '2026-10-11', 'Laundry', 'Anil', 'Completed', 'Bed linen replaced'),
(4, '2026-10-12', 'Cleaning', 'Ramesh', 'Pending', 'Deep cleaning'),
(5, '2026-10-12', 'Inspection', 'Suresh', 'Completed', 'Room inspected');

INSERT INTO ServiceCharges

(StayID, ServiceRequestID, ServiceName,
Quantity, UnitPrice, TotalAmount, ChargeDate)

VALUES

```

(1, 1, 'Room Service', 2, 500.00, 1000.00, '2026-10-10'),
(2, 2, 'Laundry', 5, 100.00, 500.00, '2026-10-11'),
(3, 3, 'Extra Bed', 1, 800.00, 800.00, '2026-10-15'),
(4, 4, 'Airport Pickup', 1, 1200.00, 1200.00, '2026-10-16'),
(5, 5, 'Room Service', 2, 450.00, 900.00, '2026-10-18');
INSERT INTO Invoices
(StayID, InvoiceDate, RoomCharges,
ServiceCharges, TaxAmount, TotalAmount, InvoiceStatus)
VALUES
(1, '2026-10-12', 5000.00, 1000.00, 720.00, 6720.00, 'Paid'),
(2, '2026-10-14', 10500.00, 500.00, 1320.00, 12320.00, 'Pending'),
(3, '2026-10-18', 18000.00, 800.00, 2256.00, 21056.00, 'Pending'),
(4, '2026-10-20', 20000.00, 1200.00, 2544.00, 23744.00, 'Pending'),
(5, '2026-10-20', 9000.00, 900.00, 1188.00, 11088.00, 'Paid');
INSERT INTO Payments
(InvoiceID, PaymentDate, Amount,
PaymentMethod, PaymentStatus, TransactionReference)
VALUES
(1, '2026-10-12', 6720.00, 'UPI', 'Completed', 'TXN10001'),
(2, '2026-10-14', 5000.00, 'Card', 'Completed', 'TXN10002'),
(3, '2026-10-18', 10000.00, 'UPI', 'Completed', 'TXN10003'),
(4, '2026-10-20', 12000.00, 'Card', 'Completed', 'TXN10004'),
(5, '2026-10-20', 11088.00, 'Cash', 'Completed', 'TXN10005');
SELECT * FROM Guests;
SELECT * FROM Rooms;
SELECT *

```

```
FROM Rooms
WHERE Status = 'Available';
SELECT * FROM Reservations;
SELECT
    r.ReservationID,
    CONCAT(g.FirstName, ' ', g.LastName) AS GuestName,
    rm.RoomNumber,
    rt.TypeName AS RoomType,
    rp.PlanName AS RatePlan,
    r.CheckInDate,
    r.CheckOutDate,
    r.NumberOfGuests,
    r.Status
FROM Reservations r
JOIN Guests g ON r.GuestID = g.GuestID
JOIN Rooms rm ON r.RoomID = rm.RoomID
JOIN RoomTypes rt ON rm.RoomTypeID = rt.RoomTypeID
JOIN RatePlans rp ON r.RatePlanID = rp.RatePlanID;
SELECT
    sr.ServiceRequestID,
    CONCAT(g.FirstName, ' ', g.LastName) AS GuestName,
    sr.ServiceType,
    sr.RequestDate,
    sr.Status,
    sr.SpecialInstructions
FROM ServiceRequests sr
```

```
JOIN Guests g
ON sr.GuestID = g.GuestID;
SELECT
    h.TaskID,
    r.RoomNumber,
    h.TaskDate,
    h.TaskType,
    h.AssignedStaff,
    h.Status
FROM HousekeepingTasks h
JOIN Rooms r
ON h.RoomID = r.RoomID;
UPDATE Guests
SET Phone = '9999999999'
WHERE GuestID = 1;
UPDATE Rooms
SET Status = 'Occupied'
WHERE RoomID = 1;
UPDATE Reservations
SET Status = 'Completed'
WHERE ReservationID = 1;
DELETE FROM Payments
WHERE PaymentID = 5;
DELETE FROM ServiceCharges
WHERE ServiceChargeID = 5;
DELETE FROM Guests
```

```
WHERE GuestID = 10;

SELECT COUNT(*) AS TotalGuests
FROM Guests;

SELECT COUNT(*) AS TotalRooms
FROM Rooms;

SELECT COUNT(*) AS AvailableRooms
FROM Rooms
WHERE Status = 'Available';

SELECT Status, COUNT(*) AS Total
FROM Reservations
GROUP BY Status;

SELECT Status, COUNT(*) AS Total
FROM Reservations
GROUP BY Status;

SELECT SUM(Amount) AS TotalPayments
FROM Payments
WHERE PaymentStatus = 'Completed';

SELECT SUM(TotalAmount) AS TotalServiceCharges
FROM ServiceCharges;

SELECT AVG(BasePrice) AS AverageRoomPrice
FROM RoomTypes;

SELECT
    CONCAT(g.FirstName, ' ', g.LastName) AS GuestName,
    r.ReservationID,
    r.CheckInDate,
    r.CheckOutDate,
```

```
        r.Status
FROM Guests g
JOIN Reservations r
ON g.GuestID = r.GuestID
ORDER BY g.GuestID;
SELECT
    i.InvoiceID,
    i.TotalAmount,
    p.PaymentDate,
    p.Amount,
    p.PaymentMethod,
    p.PaymentStatus
FROM Invoices i
JOIN Payments p
ON i.InvoiceID = p.InvoiceID;
SELECT
    CONCAT(g.FirstName, ' ', g.LastName) AS GuestName,
    sc.ServiceName,
    sc.Quantity,
    sc.UnitPrice,
    sc.TotalAmount,
    sc.ChargeDate
FROM ServiceCharges sc
JOIN Stays s
ON sc.StayID = s.StayID
JOIN Reservations r
```

```
ON s.ReservationID = r.ReservationID
JOIN Guests g
ON r.GuestID = g.GuestID;
SELECT
    s.StayID,
    CONCAT(g.FirstName, ' ', g.LastName) AS GuestName,
    rm.RoomNumber,
    s.ActualCheckIn,
    s.StayStatus
FROM Stays s
JOIN Reservations r
ON s.ReservationID = r.ReservationID
JOIN Guests g
ON r.GuestID = g.GuestID
JOIN Rooms rm
ON r.RoomID = rm.RoomID
WHERE s.StayStatus = 'Active';
SELECT
    sr.ServiceRequestID,
    CONCAT(g.FirstName, ' ', g.LastName) AS GuestName,
    sr.ServiceType,
    sr.RequestDate,
    sr.Status
FROM ServiceRequests sr
JOIN Guests g
ON sr.GuestID = g.GuestID
```

```
WHERE sr.Status = 'Pending';
```

```
SELECT
```

```
    h.TaskID,
```

```
    r.RoomNumber,
```

```
    h.TaskType,
```

```
    h.AssignedStaff,
```

```
    h.Status
```

```
FROM HousekeepingTasks h
```

```
JOIN Rooms r
```

```
ON h.RoomID = r.RoomID
```

```
WHERE h.Status = 'Pending';
```

Query that has given in presentation-2

```
SELECT
```

```
    rt.RoomTypeID,
```

```
    rt.Description,
```

```
    rt.Capacity,
```

```
    r.RoomID,
```

```
    r.RoomNumber,
```

```
    r.Status AS RoomStatus,
```

```
    rp.RatePlanID,
```

```
    rp.PlanName,
```

```
    rp.PricePerNight,
```

```
    g.GuestID,
```

```
    g.Name AS GuestName,
```

```
    g.Contact,
```

```
    g.Email,
```

res.ReservationID,
res.CheckInDate,
res.CheckOutDate,
res.Status AS ReservationStatus,
s.StayID,
s.ActualCheckIn,
s.ActualCheckOut,
sr.ServiceRequestID,
sr.ServiceType,
sr.RequestDate,
sr.Status AS ServiceStatus,
h.TaskID,
h.TaskDate,
h.TaskType,
h.Status AS HousekeepingStatus,
sc.ChargeID,
sc.ServiceDescription,
sc.Amount,
i.InvoiceID,
i.Total,
i.Status AS InvoiceStatus,
p.PaymentID,
p.PaymentDate,
p.Amount AS PaymentAmount,
p.PaymentMethod
FROM RoomTypes rt

```
JOIN Rooms r
    ON rt.RoomTypeID = r.RoomTypeID
JOIN RatePlans rp
    ON rt.RoomTypeID = rp.RoomTypeID
LEFT JOIN Reservations res
    ON r.RoomID = res.RoomID
LEFT JOIN Guests g
    ON res.GuestID = g.GuestID
LEFT JOIN Stays s
    ON res.ReservationID = s.ReservationID
LEFT JOIN ServiceRequests sr
    ON g.GuestID = sr.GuestID
LEFT JOIN HousekeepingTasks h
    ON r.RoomID = h.RoomID
LEFT JOIN ServiceCharges sc
    ON s.StayID = sc.StayID
LEFT JOIN Invoices i
    ON s.StayID = i.StayID
LEFT JOIN Payments p
    ON i.InvoiceID = p.InvoiceID
WHERE rt.RoomTypeID = 9;
```

7. RELATIONAL SCHEMA AND NORMALIZATION

7.1 Relational Schema

RoomTypes

RoomTypes(RoomTypeID, TypeName, Description, Capacity, BasePrice)

Primary Key: RoomTypeID

Rooms

Rooms(RoomID, RoomNumber, RoomTypeID, Floor, Status)

Primary Key: RoomID

Foreign Key: RoomTypeID

RatePlans

RatePlans(RatePlanID, PlanName, Description, PricePerNight, CancellationPolicy)

Primary Key: RatePlanID

Guests

Guests(GuestID, FirstName, LastName, Email, Phone, Address, IDProof)

Primary Key: GuestID

Reservations

Reservations(ReservationID, GuestID, RoomID, RatePlanID, ReservationDate, CheckInDate, CheckOutDate, NumberOfGuests, Status)

Primary Key: ReservationID

Foreign Keys: GuestID, RoomID, RatePlanID

Stays

Stays(StayID, ReservationID, ActualCheckIn, ActualCheckOut, NumberOfNights, StayStatus)

Primary Key: StayID

Foreign Key: ReservationID

ServiceRequests

ServiceRequests(ServiceRequestID, GuestID, StayID, ServiceType, RequestDate, RequestTime, Status, SpecialInstructions)

Primary Key: ServiceRequestID

Foreign Keys: GuestID, StayID

HousekeepingTasks

HousekeepingTasks(TaskID, RoomID, TaskDate, TaskType, AssignedStaff, Status, Remarks)

Primary Key: TaskID

Foreign Key: RoomID

ServiceCharges

ServiceCharges(ServiceChargeID, StayID, ServiceRequestID, ServiceName, Quantity, UnitPrice, TotalAmount, ChargeDate)

Primary Key: ServiceChargeID

Foreign Keys: StayID, ServiceRequestID

Invoices

Invoices(InvoiceID, StayID, InvoiceDate, RoomCharges, ServiceCharges, TaxAmount, TotalAmount, InvoiceStatus)

Primary Key: InvoiceID

Foreign Key: StayID

Payments

Payments(PaymentID, InvoiceID, PaymentDate, Amount, PaymentMethod, PaymentStatus, TransactionReference)

Primary Key: PaymentID

Foreign Key: InvoiceID

7.2 Normalization

Normalisation techniques have been applied in designing the database to minimise redundancy.

First Normal Form (1NF)

Values in all columns must be atomic, and there are neither repeating nor multi-valued columns.

Examples

In place of putting several phone numbers in one field, there must be only a single value in the respective column.

Second Normal Form (2NF)

Tables are in 1NF, and non-key attributes depend on the full primary key.

For instance, Guest details are in a separate table, i.e. Guests, instead of being in every reservation.

Third Normal Form (3NF)

Tables are in 2NF, but non-key attributes do not depend on any other non-key attribute.

For instance, room type details are stored in RoomTypes instead of having repeated room types in every Room record.

Thus, redundancy has been minimised.

8. DATA DICTIONARY

RoomTypes

Field	Data Type	Constraint
RoomTypeID	INT	Primary Key
TypeName	VARCHAR(50)	NOT NULL
Description	VARCHAR(255)	—
Capacity	INT	NOT NULL
BasePrice	DECIMAL(10,2)	NOT NULL

Rooms

Field	Data Type	Constraint
RoomID	INT	Primary Key
RoomNumber	VARCHAR(10)	UNIQUE
RoomTypeID	INT	Foreign Key

Floor	INT	NOT NULL
-------	-----	----------

Status	VARCHAR(20)	NOT NULL
--------	-------------	----------

RatePlans

Field	Data Type	Constraint
RatePlanID	INT	Primary Key
PlanName	VARCHAR(50)	NOT NULL
Description	VARCHAR(255)	—
PricePerNight	DECIMAL(10,2)	NOT NULL
CancellationPolicy	VARCHAR(255)	—

Guests

Field	Data Type	Constraint
GuestID	INT	Primary Key
FirstName	VARCHAR(50)	NOT NULL
LastName	VARCHAR(50)	NOT NULL
Email	VARCHAR(100)	UNIQUE
Phone	VARCHAR(15)	NOT NULL
Address	VARCHAR(255)	—
IDProof	VARCHAR(50)	—

Reservations

Field	Data Type	Constraint
ReservationID	INT	Primary Key
GuestID	INT	Foreign Key
RoomID	INT	Foreign Key

RatePlanID	INT	Foreign Key
ReservationDate	DATE	NOT NULL
CheckInDate	DATE	NOT NULL
CheckOutDate	DATE	NOT NULL
NumberOfGuests	INT	NOT NULL
Status	VARCHAR(20)	NOT NULL

Stays

Field	Data Type	Constraint
StayID	INT	Primary Key
ReservationID	INT	Foreign Key
ActualCheckIn	DATETIME	—
ActualCheckOut	DATETIME	—
NumberOfNights	INT	—
StayStatus	VARCHAR(20)	—

ServiceRequests

Field	Data Type	Constraint
ServiceRequestID	INT	Primary Key
GuestID	INT	Foreign Key
StayID	INT	Foreign Key
ServiceType	VARCHAR(100)	NOT NULL
RequestDate	DATE	NOT NULL

RequestTime	TIME	—
Status	VARCHAR(20)	NOT NULL
SpecialInstructions	VARCHAR(255)	—

HousekeepingTasks

Field	Data Type	Constraint
TaskID	INT	Primary Key
RoomID	INT	Foreign Key
TaskDate	DATE	NOT NULL
TaskType	VARCHAR(50)	NOT NULL
AssignedStaff	VARCHAR(100)	—
Status	VARCHAR(20)	NOT NULL
Remarks	VARCHAR(255)	—

ServiceCharges

Field	Data Type	Constraint
ServiceChargeID	INT	Primary Key
StayID	INT	Foreign Key
ServiceRequestID	INT	Foreign Key
ServiceName	VARCHAR(100)	NOT NULL
Quantity	INT	NOT NULL
UnitPrice	DECIMAL(10,2)	NOT NULL
TotalAmount	DECIMAL(10,2)	NOT NULL
ChargeDate	DATE	NOT NULL

Invoices

Field	Data Type	Constraint
InvoiceID	INT	Primary Key
StayID	INT	Foreign Key
InvoiceDate	DATE	NOT NULL
RoomCharges	DECIMAL(10,2)	NOT NULL
ServiceCharges	DECIMAL(10,2)	NOT NULL
TaxAmount	DECIMAL(10,2)	NOT NULL
TotalAmount	DECIMAL(10,2)	NOT NULL
InvoiceStatus	VARCHAR(20)	NOT NULL

Payments

Field	Data Type	Constraint
PaymentID	INT	Primary Key
InvoiceID	INT	Foreign Key
PaymentDate	DATE	NOT NULL
Amount	DECIMAL(10,2)	NOT NULL
PaymentMethod	VARCHAR(30)	NOT NULL
PaymentStatus	VARCHAR(20)	NOT NULL
TransactionReference	VARCHAR(100)	—

9. SQL COMANDS USED(DDL AND DML) WITH SAMPLE OUTPUTS

9.1 Create Database

```
CREATE DATABASE Hotel Management;
```

```
USE Hotel Management;
```

Sample Output

Query OK, 1 row affected

Database changed

9.2 Create Room Types Table

```
CREATE TABLE Room Types (  
RoomTypeID INT PRIMARY KEY AUTO_INCREMENT,  
Type Name VARCHAR(50) NOT NULL,  
Description VARCHAR(255),  
Capacity INT NOT NULL,  
Base Price DECIMAL(10,2) NOT NULL  
);
```

9.3 Insert Guest

```
INSERT INTO Guests  
(First Name, Last Name, Email, Phone, Address, IDProof)  
VALUES  
('Aarav', 'Sharma', 'aarav@gmail.com',  
'9876543210', 'Hyderabad', 'Aadhar');
```

Sample Output

1 row inserted successfully.

9.4 View Guests

```
SELECT * FROM Guests;
```

Sample Output

GuestID	First Name	Last Name	Email
1	Aarav	Sharma	aarav@gmail.com

9.5 Update Guest

```
UPDATE Guests
```

```
SET Phone = '9876501234'
```

```
WHERE GuestID = 1;
```

9.6 Delete Guest

```
DELETE FROM Guests
```

```
WHERE GuestID = 1;
```

The delete can only be allowed if there are no reservation records dependent upon it.

9.7 View Available Rooms

```
SELECT *
```

```
FROM Rooms
```

```
WHERE Status = 'Available';
```

9.8 Count Guests

```
SELECT COUNT(*) AS Total Guests
```

```
FROM Guests;
```

9.9 Calculate Total Payments

```
SELECT SUM(Amount) AS Total Payments
```

```
FROM Payments;
```

10. QUERIES WITH OUTPUTS

Query 1: Display all guests

```
SELECT * FROM Guests;
```

Output: Displays all registered guests.

Query 2: Display available rooms

```
SELECT RoomID, Room Number, RoomTypeID, Floor, Status
```

```
FROM Rooms
```

```
WHERE Status = 'Available';
```

Output: Displays rooms that are currently available.

Query 3: Display reservation details

```
SELECT
r. ReservationID,
CONCAT(g. First Name, ' ', g. Last Name) AS Guest Name,
rm. Room Number,
rt. Type Name AS Room Type,
rp. Plan Name AS Rate Plan,
r. CheckInDate,
r. Check Out Date,
r. NumberOfGuests,
r. Status
FROM Reservations r
JOIN Guests g ON r. GuestID = g. GuestID
JOIN Rooms rm ON r. RoomID = rm. RoomID
JOIN Room Types rt ON rm. RoomTypeID = rt. RoomTypeID
JOIN Rate Plans rp ON r. RatePlanID = rp. RatePlanID;
```

Sample Output

ReservationID	Guest Name	Room Number	Room Type	Rate Plan	Status
---------------	------------	-------------	-----------	-----------	--------

1	Aarav Sharma	101	Deluxe Standard	Confirmed
---	--------------	-----	-----------------	-----------

2	Ananya Reddy	205	Suite Premium	Confirmed
---	--------------	-----	---------------	-----------

Query 4: Display guest service requests

```
SELECT
sr. ServiceRequestID,
CONCAT(g. First Name, ' ', g. Last Name) AS Guest Name,
sr. Service Type,
sr. Request Date,
```

sr. Status

FROM Service Requests sr

JOIN Guests g

ON sr. GuestID = g. GuestID;

Query 5: Count reservations by status

SELECT Status, COUNT(*) AS Total Reservations

FROM Reservations

GROUP BY Status;

Sample Output

Status Total Reservations

Confirmed 6

Completed 2

Cancelled 2

Query 6: Calculate total revenue

SELECT SUM(Amount) AS Total Revenue

FROM Payments

WHERE Payment Status = 'Completed';

Query 7: Display invoice and payment details

SELECT

i. InvoiceID,

i. Total Amount,

p. Payment Date,

p. Amount,

p. Payment Method,

p. Payment Status

FROM Invoices i

JOIN Payments p

ON i. InvoiceID = p. InvoiceID;

Query 8: Display pending housekeeping tasks

SELECT *

FROM Housekeeping Tasks

WHERE Status = 'Pending';

11. UI DESIGN AND SCREENSHOTS

It offers an easy to use and simple hotel administration platform.

Main Screens

1. Dashboard

Displays:

Total Guests

Total Rooms

Available Rooms

Total Reservations

Active Stays

Pending Services

Pending Housekeeping Tasks

Total Payments

Screenshot: Insert Dashboard screenshot here.

2. Guests Screen

Displays all guest records.

Functions:

Add Guest

View Guests

Search Guest

Edit Guest

Delete Guest

Screenshot: Insert Guests screen here.

3. Rooms Screen

Displays:

Room Number

Room Type

Floor

Status

Screenshot: Insert Rooms screen here.

4. Reservations Screen

Exhibits reserving particulars with visitor, room and charge plan info.

Screenshot: Insert Reservations screen here.

5. Service Requests Screen

Provides the ability for staff to add and review guest service requests.

Screenshot: Insert Service Requests screen here.

6. Invoices and Payments

Displays invoice and payment records.

Screenshot: Insert Invoice/Payment screen here.

Required Screenshots for Presentation-III

As per the faculty instruction, provide screens shots of:

Insert screen before insertion.

Insert screen after insertion.

View records.

Delete confirmation.

Records before deletion.

Records after deletion.

12. IMPLEMENTATION DETAILS

12.1 Technology Stack

The application is implemented using:

Frontend

HTML

CSS

JavaScript

Backend

Node.js

Express.js

Database

MySQL

Development/Deployment

Replit

12.2 Database Connectivity

The web app is connected to MySQL through the backend.

Create a MySQL connection pool The mysql2 package allows you to create a connection pool.

Example:

```
const mysql = require('mysql2');
const db = mysql.createConnection({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME
});
db.connect((err) => {
```

```

if (err) {
  console.error('Database connection failed:', err);
} else {
  console.log('Connected to MySQL database');
}
});

```

12.3 Insert Operation

Example backend query:

```

const sql =
INSERT INTO Guests
(First Name, Last Name, Email, Phone, Address, IDProof)
VALUES (?, ?, ?, ?, ?, ?)
;
db.query(sql,
[first Name, last Name, email, phone, address, idProof],
(err, result) => {
  if (err) {
    console.error(err);
  } else {
    console.log('Guest inserted successfully');
  }
}
);

```

12.4 View Operation

```

db.query(
'SELECT * FROM Guests',

```

```
(err, results) => {  
  if (err) {  
    console.error(err);  
  } else {  
    console.log(results);  
  }  
}  
);
```

12.5 Delete Operation

Given, const sql='DELETE FROM Guests WHERE GuestID=?';

```
db.query(sql, [guestID], (err, result) => {  
  if (err) {  
    console.error(err);  
  } else {  
    console.log('Guest deleted successfully');  
  }  
});
```

12.6 Database Connectivity Flow

User Interface

HTML / CSS / JavaScript

Node.js + Express.js

MySQL Connection

Hotel Management Database

Tables and Records

13. TESTING - TEST CASES AND RESULTS

Tested application to ensure database connection and CRUD functionality.

Test Case Operation Expected Result

Pass: TC02 Guest should be added. InsertTC03 Name:Name is displayed on guest list Pass

TC02 View Guests View Pass Guests View all guest records displayed Pass

TC03 Update Guest details updated Pass

TC04 Delete Guest record deleted Pass

TC05 Add Room inserted successfully Pass

TC06 View Rooms Room records displayed Pass

TC07 Add Reservation inserted Pass

TC08 View Reservation displayed Pass

TC09 Search Guest Matching guest displayed Pass

TC10 Add Service Request Service request inserted Pass

TC11 View Service Requests Records displayed Pass

TC12 Add Payment inserted Pass

TC13 Delete Payment deleted Pass

TC14 Invalid Email Validation error displayed Pass

TC15 Invalid Foreign Key Operation rejected Pass

TC16 SQL JOIN Combined information displayed Pass

Testing Conclusion

The test cases indicate that the application has carried out the required database operations successfully. Insert, delete, view, update, search and SQL query operations have been tested.

14. CONCLUSION AND FUTURE ENHANCEMENTS

14.1 Conclusion

The Hotel Reservation and Guest Service Management System is a clear example of how this DBMS may be applied to a real hotel management problem.

The system offers a relational, single source database for managing room types, rooms, rate plans, guests, reservations, stays, service requests, housekeeping tasks, service charges, invoices and payments.

The tables are related through the use of primary keys and foreign keys and the data in the tables is guaranteed using data integrity SQL commands insert, query, update, delete and analyze the data.

In the user interface, hotel staff can add records, view records, and delete records. So, the project fulfills the fundamental needs of a DBMS hotel management application.

14.2 Future Enhancements

The system can be enhanced with:

Online hotel booking.

Guest login and registration.

Admin authentication.

Online payment gateway.

Email/SMS reservation notifications.

Automatic room availability calculation.

Customer feedback and rating system.

Mobile application.

Advanced hotel analytics dashboard.

Automatic invoice generation.

QR-based check-in.

Integration with external booking platforms.

Role-based access control.

Cloud database deployment.

Automated reports and charts.

15. REFERENCES

Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, Pearson.

Korth, H. F., Silberschatz, A. and Sudarshan, S., Database System Concepts, McGraw-Hill.

MySQL Documentation - MySQL Reference Manual.

Node.js Documentation.

Express.js Documentation.

W3Schools - SQL and Database Concepts.

MDN Web Docs - HTML, CSS and JavaScript.

Replit Documentation.

16. APPENDIX - GITHUB REPOSITORY LINK

GitHub Repository:

Paste your GitHub repository link here.

Live Application:

Paste your Replit deployed application link here.

Appendix Contents

The repository should contain:

Source code

SQL database file

Table creation commands

Insert statements

SELECT queries

UPDATE queries

DELETE queries

ER diagram

Screenshots

README file

Project documentation

This SQL file contains the complete database creation script and sample data in order to recreate the database.

GitHub Link: <https://github.com/kavurirohan-prog/hotel-reservation-guest-service-dbms>

Live Application: <https://hotel-operations-hub--kavurirohan.replit.app/>